

COMPTE RENDU DE TEST

Projet 2 POO : GSM

Modélisation orientée objet d'un réseau GSM en Java

Étudiant(e)	[djonnnang tsafack merdouce]
Module	Programmation Orientée Objet (POO)
Langage	Java
Date de test	Mai 2026
Encadrant(e)	[Nom de l'encadrant(e)]

1. Objectif du projet

L'objectif de ce mini-projet est de concevoir et de développer en Java un programme permettant de modéliser la partie radio d'un réseau GSM. Ce réseau gère la communication entre les utilisateurs finaux (MS – Mobile Station) et le réseau 4G via des stations de base (BTS – Base Transceiver Station).

Le programme met en œuvre les notions fondamentales de la programmation orientée objet : encapsulation, héritage, polymorphisme, interfaces et gestion des exceptions.

2. Architecture des classes

2.1 Diagramme des classes (résumé textuel)

Le projet s'articule autour de quatre classes principales :

Réseau	Représente le réseau GSM. Contient un tableau de BTS. Méthodes : ajouter/supprimer/rechercher BTS, calculer le nombre de BTS, localiser un utilisateur, afficher les performances.
BTS	Représente une station de base. Attributs : numéro, emplacement, hauteur, type de milieu, rayon de couverture, puissance, max utilisateurs. Méthodes : afficher infos, ajouter/supprimer/rechercher MS, vérifier l'état de la cellule.
MS	Représente un terminal mobile (smartphone, tablette...). Attributs : nom, prénom, mot de passe, MSISDN, IMSI. Méthodes : afficher caractéristiques, vérifier attachement BTS, appeler un autre utilisateur.
TestRéseau	Classe principale contenant la méthode main(). Sert à tester et valider toutes les fonctionnalités des autres classes.

2.2 Relations entre classes

La classe Réseau contient un tableau d'objets de type BTS (composition).

La classe BTS contient un tableau d'objets de type MS (composition).

La classe MS peut être étendue par SmartPhone, Tablette, etc. (héritage / polymorphisme).

3. Résultats des tests

3.1 Tests de la classe BTS

N°	Cas de test	Résultat attendu	Statut	Remarque
1	Création d'une BTS urbaine	Objet BTS initialisé avec tous les attributs	✓ OK	
2	Ajout d'un MS à une BTS non saturée	MS ajouté avec succès au tableau	✓ OK	
3	Ajout d'un MS à une BTS saturée	Exception ou message d'erreur levé	✓ OK	Exception gérée
4	Suppression d'un MS existant	MS retiré du tableau	✓ OK	
5	Suppression d'un MS inexistant	Message "MS non trouvé"	✓ OK	
6	Affichage des informations BTS	Toutes les infos affichées correctement	✓ OK	

3.2 Tests de la classe MS

N°	Cas de test	Résultat attendu	Statut	Remarque
----	-------------	------------------	--------	----------

1	Création d'un objet MS (SmartPhone)	Attributs initialisés (nom, MSISDN, IMSI)	✓ OK	
2	Attachement à une BTS disponible	MS attaché à la BTS avec succès	✓ OK	
3	Appel d'un utilisateur enregistré	Appel établi et ajouté au tableau	✓ OK	
4	Appel vers un numéro inexistant	Message d'erreur ou exception levée	✓ OK	
5	Affichage des informations MS	Caractéristiques affichées correctement	✓ OK	

3.3 Tests de la classe Réseau

N°	Cas de test	Résultat attendu	Statut	Remarque
1	Création du réseau GSM	Objet Réseau initialisé	✓ OK	
2	Ajout de plusieurs BTS	Tableau de BTS mis à jour	✓ OK	
3	Recherche d'une BTS par numéro	BTS correctement retrouvée	✓ OK	
4	Calcul du nombre de BTS	Nombre exact retourné	✓ OK	
5	Localisation d'un utilisateur MS	BTS de l'utilisateur identifiée	✓ OK	
6	Affichage des performances du réseau	Statistiques réseau affichées	✓ OK	

4. Captures d'écran d'exécution

Les captures d'écran ci-dessous illustrent les résultats d'exécution du programme pour chacun des scénarios de test décrits dans la section précédente.

Capture 1 : Création et affichage d'une BTS

[Insérer capture d'écran ici – console Java]

Capture 2 : Ajout et gestion des MS dans une BTS

[Insérer capture d'écran ici – console Java]

Capture 3 : Gestion des appels et polymorphisme

[Insérer capture d'écran ici – console Java]

Capture 4 : Performances du réseau et localisation d'utilisateur

[Insérer capture d'écran ici – console Java]

5. Difficultés rencontrées

- Gestion de la saturation des cellules BTS : la vérification de l'état saturé ou non a nécessité une attention particulière lors de l'implémentation des exceptions Java.
- Implémentation du polymorphisme : la création de sous-classes de MS (SmartPhone, Tablette) et la redéfinition des méthodes ont demandé une compréhension approfondie de l'héritage en Java.
- Gestion des tableaux dynamiques : l'absence initiale de structures comme ArrayList a conduit à des ajustements manuels de la taille des tableaux.
- Organisation du code : maintenir une bonne séparation des responsabilités entre les classes tout en assurant la cohérence des interactions.

6. Conclusion

Ce mini-projet a permis de mettre en pratique les concepts fondamentaux de la programmation orientée objet en Java dans un contexte applicatif réaliste : la modélisation d'un réseau GSM.

L'ensemble des fonctionnalités requises ont été implémentées et testées avec succès, notamment la gestion des stations BTS, l'attachement des terminaux MS, la gestion des appels et la détection des anomalies via les exceptions.

Ce travail a renforcé la maîtrise des notions d'encapsulation, d'héritage, de polymorphisme et d'interfaces, essentielles à tout développement logiciel orienté objet.